Luca Danisi **17-4-2026**

`Buffer overflow`

1 - 🎯 Obiettivo :

Lo scopo di questo laboratorio è capire **cos'è un Buffer Overflow (BOF)** e osservarlo in azione su un piccolo programma scritto in linguaggio C.

Un Buffer Overflow è una vulnerabilità che si verifica quando un programma accetta un input dall'utente **senza controllare quanto è lungo**. Se l'input supera lo spazio disponibile nel buffer (cioè il "contenitore" che dovrebbe accoglierlo), i caratteri in eccesso vanno a finire in zone di memoria **non autorizzate**, causando un errore di tipo *segmentation fault.*

Nello specifico, in questo esercizio andremo a:

1. Scrivere un programma in C volutamente vulnerabile, che dichiara un buffer di 10 caratteri e legge l'input utente senza alcun controllo.
2. Compilarlo ed eseguirlo normalmente per verificarne il funzionamento corretto con input brevi.
3. Provocare il Buffer Overflow fornendo al programma un input più lungo del buffer, ricevendo il segnale di *segmentation fault*.
4. Modificare il codice aumentando la dimensione del buffer a 30 caratteri, per rispondere alla domanda: **"aumentare lo spazio del buffer basta a risolvere la vulnerabilità?"**

L'obiettivo finale è quindi non solo riprodurre l'errore, ma capire **qual è la vera causa del BOF** e perché aumentare la dimensione del buffer, da solo, **non è una soluzione di sicurezza**.

🧪 Laboratorio

🐉 💀 **Kali Linux**: su rete interna (192.168.50.100)

2 - Strumenti utilizzati :

Macchina virtuale **Kali Linux** (IP `192.168.50.100`):

- **Terminale (Bash/Zsh)** — Tutto il laboratorio è stato svolto da riga di comando.

- **GCC (GNU Compiler Collection)** — il compilatore standard per il linguaggio C su Linux. Serve a trasformare il codice sorgente `.c` (leggibile dall'uomo) in un file eseguibile (leggibile dalla macchina).
- **Linguaggio C** — linguaggio di programmazione in cui è stato scritto il programma volutamente vulnerabile. Il C è particolarmente adatto per studiare i Buffer Overflow perché, a differenza di linguaggi più moderni, **non effettua controlli automatici sui limiti dei buffer**, lasciando al programmatore la responsabilità di gestire la memoria.

3 – Risultati:

✅ Test 1 — Programma con buffer da 10 caratteri, input corto

Ho scritto il programma BOF.c dichiarando un buffer da 10 caratteri e l'ho compilato con gcc -g BOF.c -o BOF. Al primo lancio ho inserito come input la stringa esercizio (9 caratteri).

Risultato: il programma ha stampato regolarmente Nome utente inserito: esercizio e si è chiuso senza errori. Tutto come previsto: 9 caratteri stanno comodi in un buffer da 10.

❌ Test 2 — Programma con buffer da 10 caratteri, input lungo

Ho rilanciato lo stesso programma e questa volta ho inserito una stringa molto più lunga: `eserciziomoooltopiùlungo` (24 caratteri).

Risultato: il programma ha comunque stampato Nome utente inserito: eserciziomoooltopiùlungo, ma subito dopo è comparso l'errore:

zsh: segmentation fault ./BOF

Questo è il classico segnale del **Buffer Overflow**: i caratteri in eccesso hanno sovrascritto aree di memoria non autorizzate e il sistema operativo ha terminato il processo per proteggersi.

✅ Test 3 — Programma con buffer da 30 caratteri, input corto

Ho modificato il codice sorgente cambiando char buffer [10]; in char buffer [30];, ho ricompilato con gcc -g BOF.c -o BOF e ho rilanciato il programma inserendo sempre la stringa esercizio.

Risultato: tutto regolare, il programma ha risposto correttamente senza errori.

❌ Test 4 — Programma con buffer da 30 caratteri, input lungo

Sempre con buffer da 30, ho inserito una stringa ancora più lunga: eserciziomoooooltopiulungodiquellodiprima (40 caratteri).

Risultato:

zsh: segmentation fault ./BOF

Il **Buffer Overflow si è ripresentato**, esattamente come nel Test 2. Aumentare la dimensione del buffer da 10 a 30 non ha risolto niente: ha solo spostato la soglia oltre la quale il problema si manifesta.

4 – Conclusioni:

Aumentare lo spazio del buffer non risolve la vulnerabilità, **la sposta soltanto**: alza la soglia oltre la quale il problema si manifesta, ma la causa resta esattamente la stessa.

La vera causa **non è la dimensione del buffer**, ma il fatto che il programma **legge l'input senza alcun controllo sulla sua lunghezza**. Finché il programma accetta qualsiasi quantità di caratteri senza verificare se c'è abbastanza spazio, un attaccante può sempre inserire un input più lungo del buffer e scatenare l'overflow.

Per eliminare davvero la vulnerabilità bisogna **controllare l'input prima di scriverlo nel buffer, definendo la lunghezza max di caratteri accettati.**

Svolgimento Tecnico

Step 1 — Apertura del terminale e spostamento sul Desktop

Ho aperto il terminale e mi sono spostato sul Desktop con il comando:

“cd /home/kali/Desktop”

Step 2 — Creazione del file sorgente BOF.c

Ho creato il file BOF.c usando l'editor di testo **nano**:

“nano BOF.c”

All'interno di nano ho scritto il seguente codice;

```
#include <stdio.h>
int main () {
char buffer [10];
printf ("Si prega di inserire il nome utente:");
scanf ("%s", buffer);
printf ("Nome utente inserito: %s\n", buffer);
return 0;
}
```

Il programma dichiara un buffer di 10 caratteri, chiede all'utente di inserire un nome utente tramite scanf("%s", buffer) — funzione che non controlla la lunghezza dell'input — e poi lo stampa a video.

Step 3 — Compilazione del programma

Ho compilato il sorgente con il comando:

“gcc -g BOF.c -o BOF”

Ovvero con **GCC, abbiamo indicato il sorgente BOF.c, e chiesto di tradurlo in linguaggio macchina e salvare il risultato in un file eseguibile BOF.**

Step 4 — Primo test (buffer da 10, input corto)

Ho eseguito il programma:

“./BOF”

Alla richiesta del nome utente ho inserito la stringa esercizio (9 caratteri). Il programma ha stampato correttamente:

Step 5 — Secondo test (buffer da 10, input lungo) — Buffer Overflow!

Ho rilanciato **“./BOF”** e questa volta ho inserito una stringa molto più lunga: eserciziomoooltopiùlungo (25 caratteri). Risultato:

Il programma è andato in **segmentation fault**: i caratteri in eccesso hanno sovrascritto zone di memoria non autorizzate e il sistema operativo ha terminato forzatamente il processo. **Buffer Overflow**.

Step 6 — Modifica del codice (buffer da 30)

Ho modificato il sorgente cambiando la dichiarazione del buffer:

- **Da:** char buffer [10];
- **A:** char buffer [30];

```
#include <stdio.h>
int main () {
char buffer [30];
printf ("Si prega di inserire il nome utente:");
scanf ("%s", buffer);
printf ("Nome utente inserito: %s\n", buffer);
return 0;
}
```

Step 7 — Ricompilazione

Dato che il codice era stato modificato, ho ricompilato il programma :

“gcc -g BOF.c -o BOF”

Step 8 — Terzo test (buffer da 30, input corto)

Ho rilanciato **“./BOF”** e inserito di nuovo esercizio. Il programma ha funzionato senza errori.

Step 9 — Quarto test (buffer da 30, input lungo) — Buffer Overflow di nuovo!

Ho rilanciato **“./BOF”** e inserito una stringa ancora più lunga: eserciziomoooooltopiulungodiquellodiprima (~40 caratteri). Risultato:

zsh: segmentation fault ./BOF

Il **Buffer Overflow si è ripresentato**. Questo dimostra chiaramente che aumentare la dimensione del buffer **non risolve la vulnerabilità**, ma sposta solo la soglia oltre la quale il problema si manifesta.

Screenshot

Figure 1 – Programma

```
  GNU nano 8.7.1
#include <stdio.h>

int main () {

char buffer [10];

printf ("Si prega di inserire il nome utente:");
scanf ("%s", buffer);

printf ("Nome utente inserito: %s\n", buffer);

return 0;
}
```

Figure 2 – Buffer over

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente:esercizio
Nome utente inserito: esercizio

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente:eserciziomoooooltopiulungodiquellodiprima
Nome utente inserito: eserciziomoooooltopiulungodiquellodiprima
zsh: segmentation fault  ./BOF

┌──(kali㉿kali)-[~/Desktop]
└─$
```